

Sample Technical Assessment

Node.js + React + PostgreSQL + AI Data Assistant

Suggested time: 2–2.5 hours **AI coding tools:** Allowed **Goal:** Build a working end-to-end solution

Scenario

You are building an internal sales assistant. The company provides a sales dataset and wants a chat interface where users can ask natural-language questions such as “*What was our revenue last month?*” or “*Which products generated the most revenue?*”. The assistant must answer using the provided data, but raw customer/order data must not be sent directly to the LLM.

Input dataset

Use the supplied **sales_data.csv** or **sales_data.json**. The dataset is intentionally denormalized. Your application should decide how it should be represented in PostgreSQL.

Field	Meaning
order_id	Unique order identifier
order_date	UTC timestamp
customer_id	Customer identifier
customer_name / email	Customer information
country	Customer country
product_id	Product identifier
product_name	Product name
category	Product category
quantity	Units purchased
unit_price	Price before discount
discount_pct	Discount percentage
total_amount	Final line/order amount
status	completed / pending / cancelled / refunded
payment_method	Payment method
channel	web / mobile

Core requirements

- **1. Import the dataset into PostgreSQL.** Design a sensible relational schema. Use appropriate data types, primary keys, foreign keys, constraints, and indexes where justified.
- **2. Build a Node.js API.** Provide an endpoint such as `POST /api/chat` that accepts a user's natural-language question.
- **3. Build a React chat UI.** Show conversation history, loading state, errors, and the assistant response.
- **4. Add an AI layer.** The LLM should help interpret the user's question, but it must not receive the raw dataset.
- **5. Keep database access under application control.** The backend should validate AI output before executing database queries.

- **6. Return useful answers.** The assistant should handle questions it can answer and gracefully reject questions outside the available data.

Recommended AI/data boundary

A strong solution can use a flow similar to:

Step	Action	Component
1	User question	React
2	Question + database/schema metadata	Node.js → LLM
3	Structured intent / query plan	LLM → Node.js
4	Validate the structured output	Node.js
5	Execute parameterized, controlled SQL	Node.js → PostgreSQL
6	Return only the minimum result needed	PostgreSQL → Node.js
7	Generate or format final answer	Node.js / LLM
8	Display answer	Node.js → React

Example questions your assistant should support

- How many completed orders were placed?
- What was the total revenue in March 2026?
- What are the top 5 products by revenue?
- Which country generated the most revenue?
- How many orders did each customer place?
- Which customers have never placed a completed order?
- Compare revenue between web and mobile channels.
- What was the average order value by month?

Security / engineering requirements

- Do not send the entire CSV/JSON dataset to the LLM.
- Do not put database credentials in the React application.
- Use parameterized SQL queries.
- Do not blindly execute arbitrary SQL returned by the LLM.
- Validate structured LLM output at runtime.
- Prefer a read-only database user for AI-generated data access.
- Limit returned rows/results where appropriate.
- Do not expose secrets in the API response or client bundle.
- Handle invalid questions, empty results, database errors, and AI failures gracefully.

Suggested PostgreSQL topics to demonstrate

PRIMARY KEY, FOREIGN KEY / REFERENCES, UNIQUE, NOT NULL, CHECK, appropriate numeric/time types, JOINS, GROUP BY, HAVING, aggregates, indexes, parameterized queries, and EXPLAIN where useful.

Suggested API

POST /api/chat

Request: { "message": "What were our top 5 products last month?" }

Response: { "answer": "... " }

What to prioritize if time runs short

- Get one complete vertical slice working before adding lots of features.
- Prioritize correct data access and the AI/database boundary over visual polish.
- Keep the architecture understandable and explainable.
- Add a few meaningful tests rather than spending the remaining time on test infrastructure.
- Finish with a short README explaining setup, architecture, assumptions, and security decisions.

Optional stretch goals

- Streaming assistant responses.
- Conversation history persisted in PostgreSQL.
- Query/result provenance shown in the UI without exposing raw sensitive rows.

- A small dashboard for common aggregate questions.
- Schema-aware structured tool/function calling.
- Automated tests for prompt-injection and unsafe-query cases.

Assessment mindset: The goal is not to build the largest system. Build a small, working, secure system and be prepared to explain your tradeoffs.